From Requirements to Implementations…

We know what to build, but how? How to we meet the quality goals?



Software architecture: Key design decisions, made early in the development, focusing on key product qualities

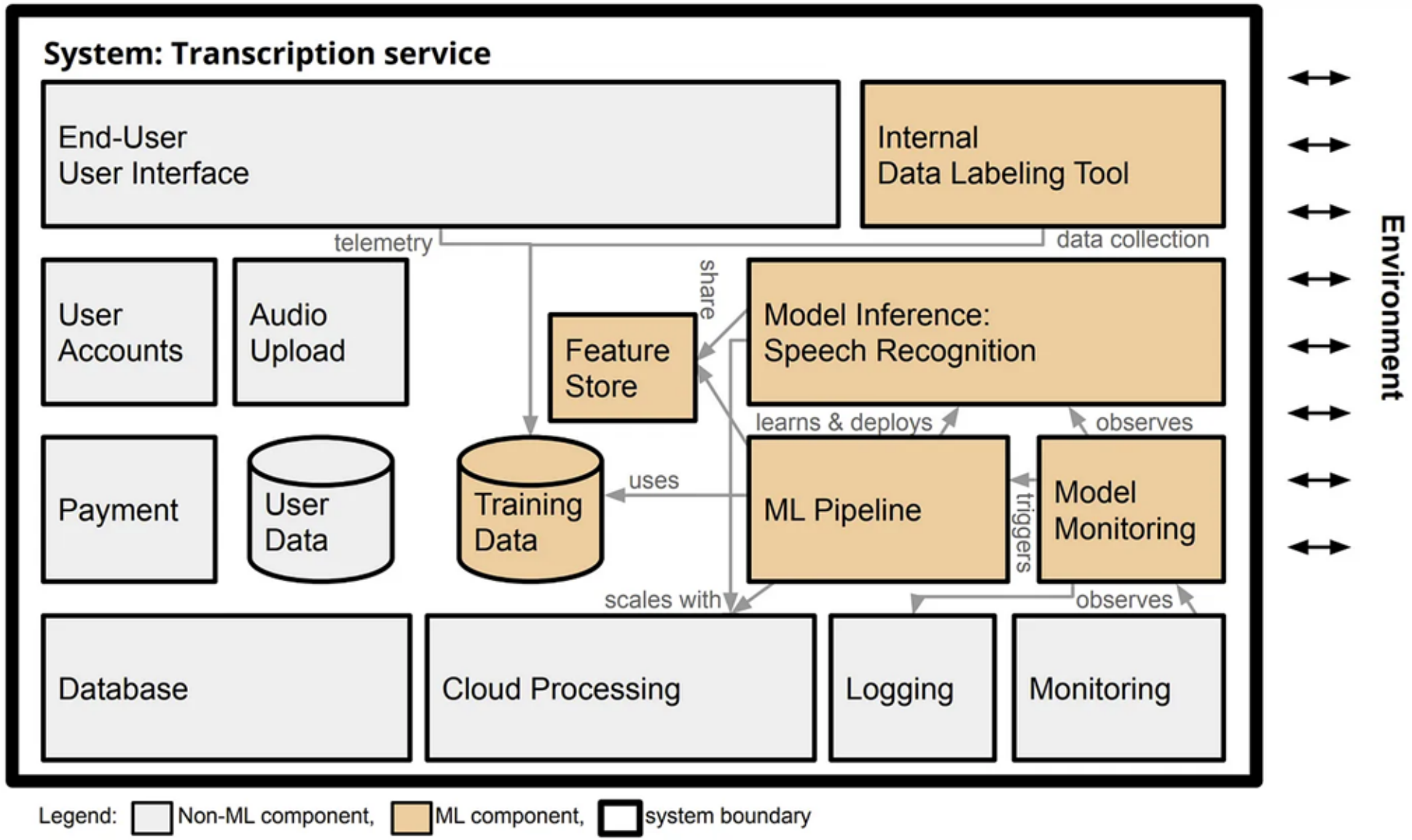
Architectural decisions are hard to change later

Architecture Decisions: Examples

- What are the major components in the system? What does each component do?
- Where do the components live? Monolithic vs microservices?
- How do components communicate to each other? Synchronous vs asynchronous calls?
- What API does each component publish? Who can access this API?
- Where does the ML inference happen? Client-side or server-side?
- Where is the telemetry data collected from the users stored?
- How large should the user database be? Centralized vs decentralized?
- …and many others

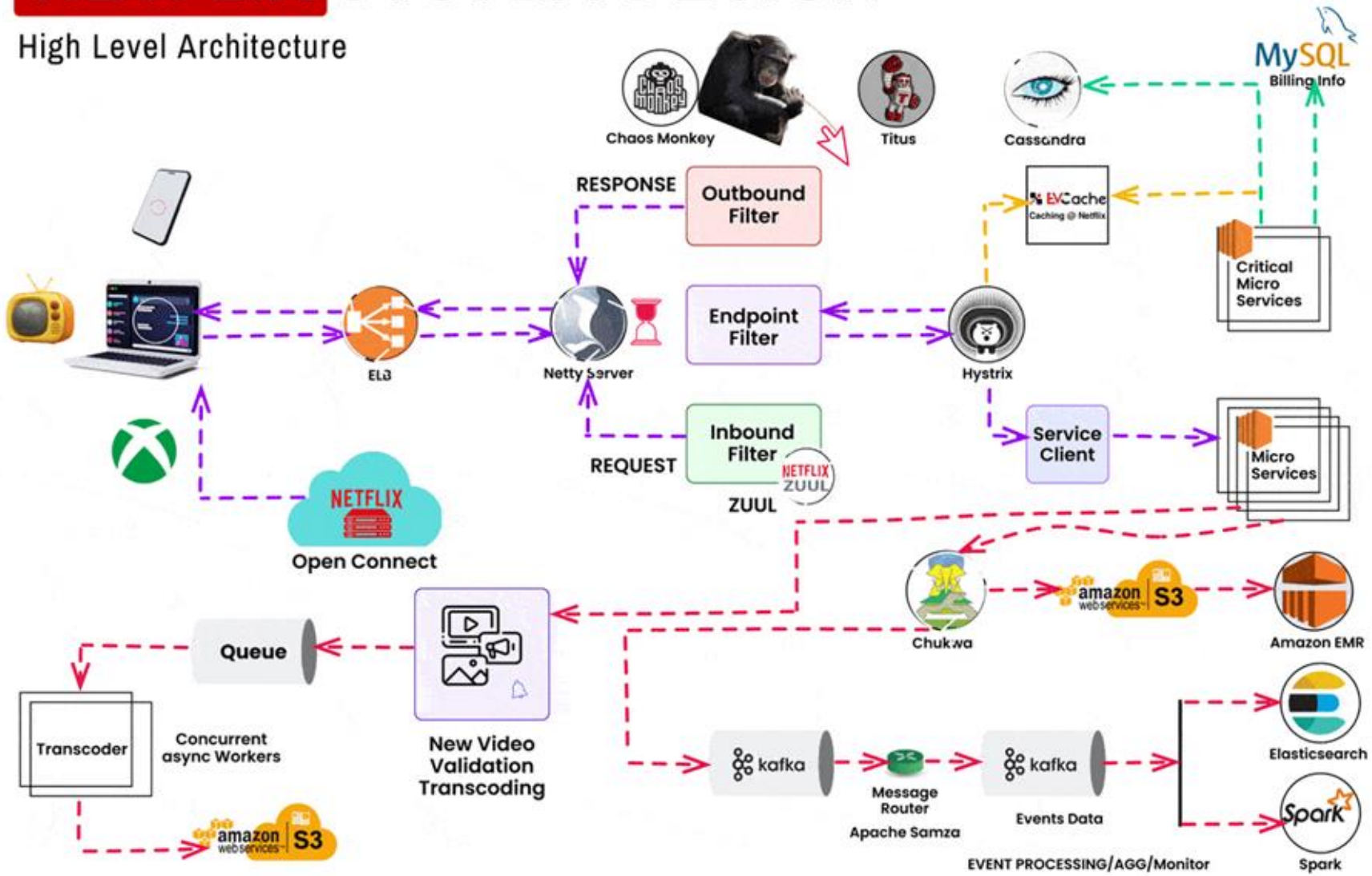
Software Architecture

*Architecture represents the set of **significant design** decisions that shape the form and the function of a system, where **significant** is measured by cost of change. -- [Grady Booch, 2006]*


System: Transcription service
End-User
User Interface
Internal
Data Labeling Tool
telemetry
data collection
User
Accounts
Audio
Upload
share
Feature
Store
Model Inference:
Speech Recognition
learns & deploys
observes
Payment
User
Data
Training
Data
uses
ML Pipeline
triggers
Model
Monitoring
scales with
observes
Database
Cloud Processing
Logging
Monitoring
Environment
Legend:
Non-ML component,
ML component,
system boundary

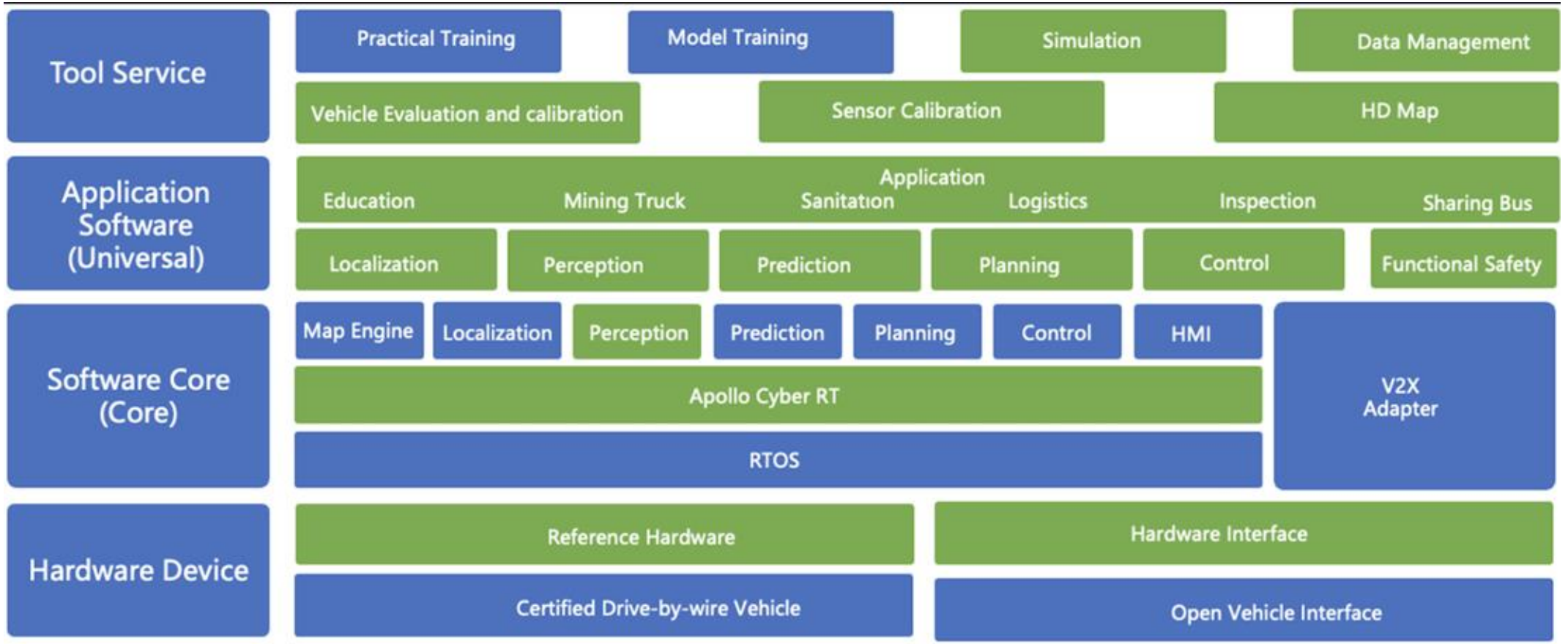
NETFLIX SYSTEM DESIGN

High Level Architecture

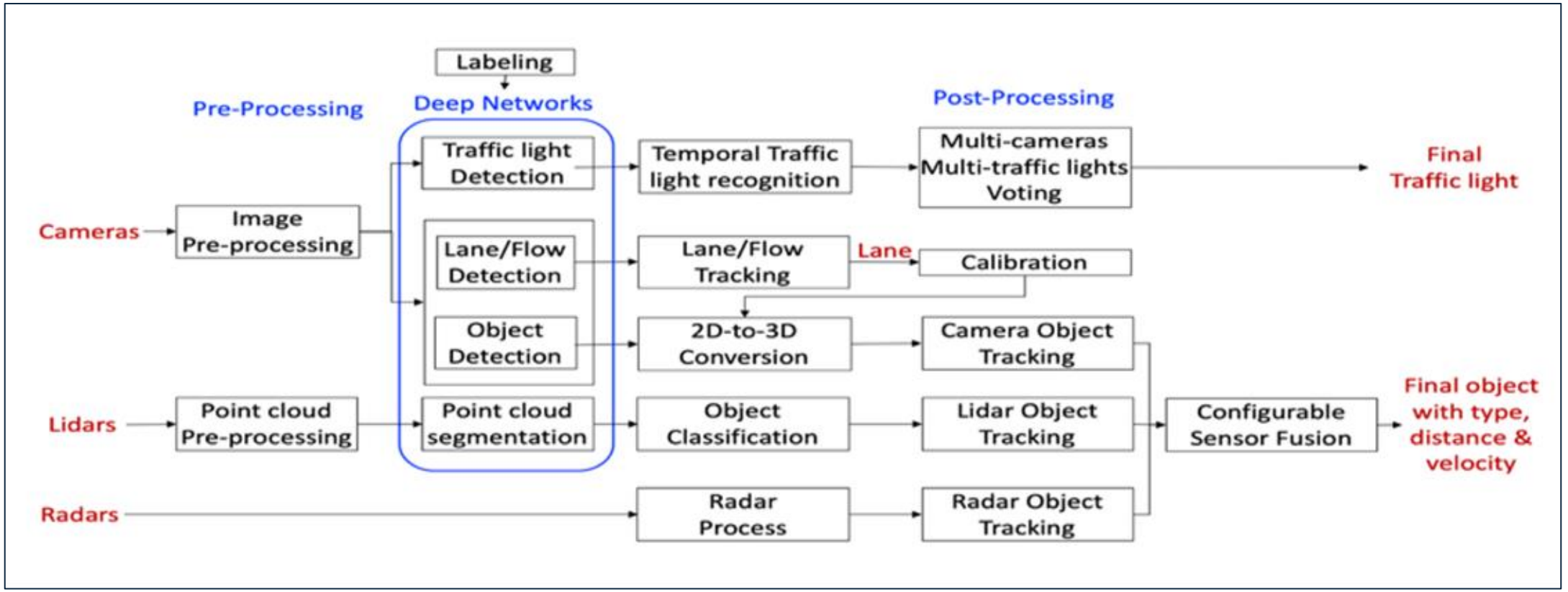


Apollo: An open source self-driving car

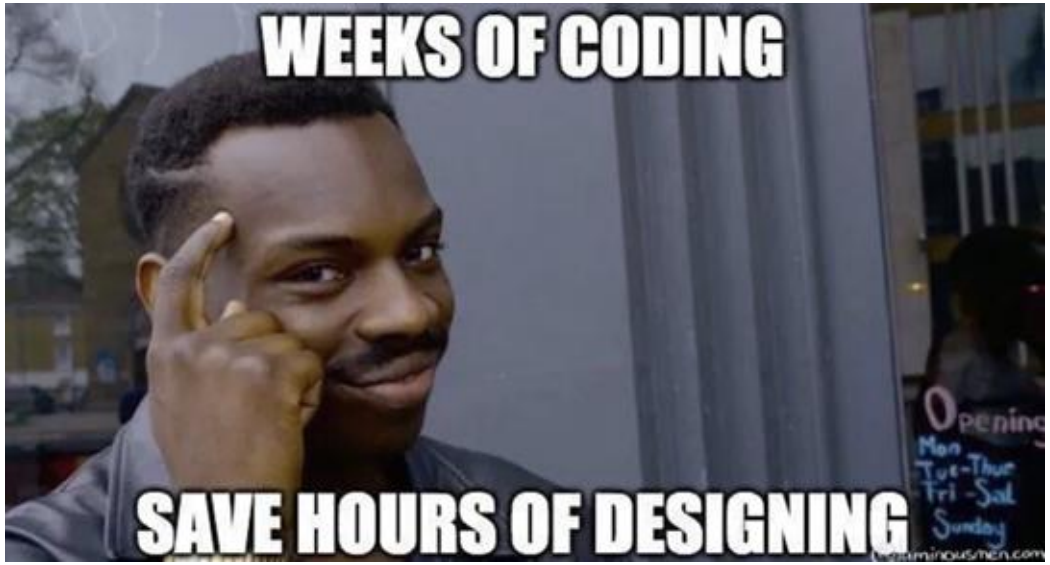
https://github.com/ApolloAuto/apollo



Apollo’s perception component



How much Architecture/Design?



Software Engineering Theme: *Think before you code*

Like requirements: Slower initially, but upfront investment can prevent problems later and save overall costs

-> Focus on most important qualities early, but leave flexibility

Why Architecture?

Represents earliest design decisions.

Aids in **communication** with stakeholders: Shows them “how” at a level they can understand, raising questions about whether it meets their needs

Defines **constraints** on implementation: Design decisions form “load-bearing walls” of application

Dictates **organizational structure**: Teams work on different components (Conway’s law)

Inhibits or enables **quality attributes**: Similar to design patterns

Supports **predicting** cost, quality, and schedule: Typically by predicting information for each component

Aids in software **evolution**: Reason about cost, design, and effect of changes

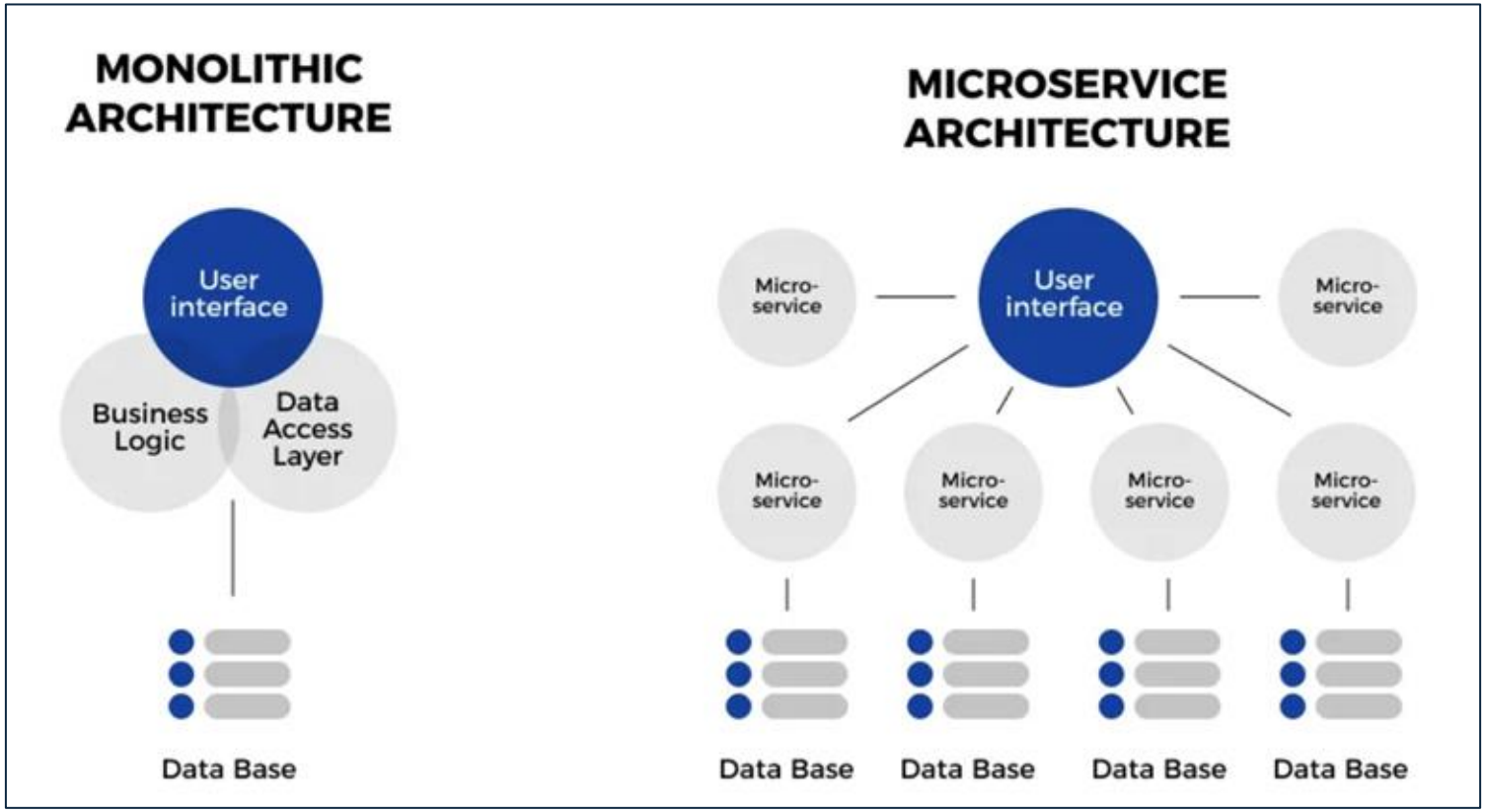
Quality Requirements Drive Architecture Design

Driven by requirements, identify most important qualities

Examples:

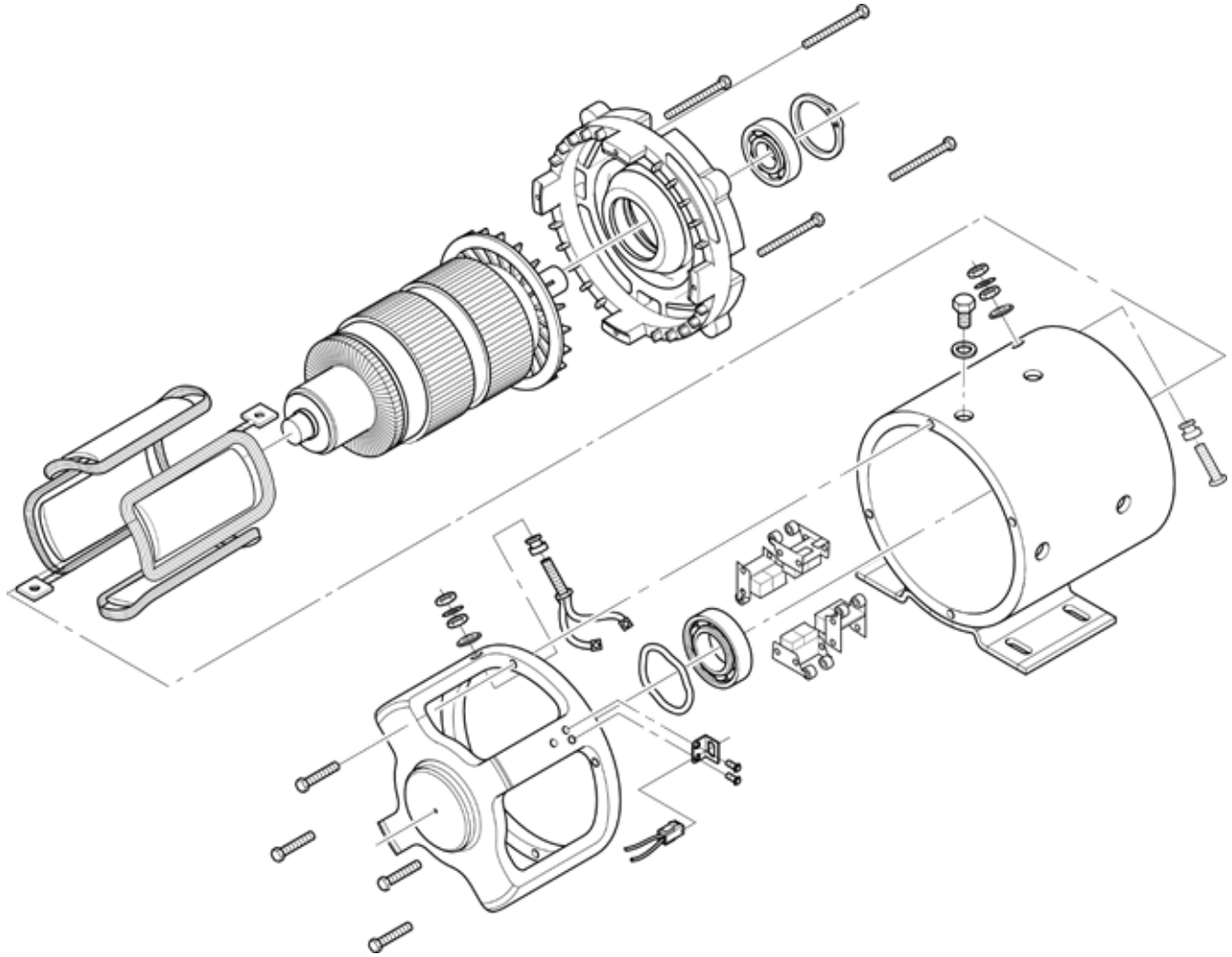
- Development cost, operational cost, time to release
- Scalability, availability, response time, throughput
- Security, safety, usability, fairness
- Ease of modifications and updates
- ML: Accuracy, ability to collect data, training latency

Architecture Design Involves Quality Trade-offs



Q. What are quality trade-offs between the two?

Codifying Design Knowledge



Common Components in ML-based Systems

- **Model inference service**: Uses model to make predictions for input data
- **ML pipeline**: Infrastructure to train/update the model
- **Monitoring**: Observe model and system
- **Data sources**: Manual/crowdsourcing/logs/telemetry/…
- **Data management**: Storage and processing of data, often at scale
- **Feature store**: Reusable feature engineering code, cached feature computations

Common System-Wide Design Challenges

Separating concerns, understanding interdependencies

- e.g., anticipating/breaking feedback loops, conflicting needs of components

Facilitating experimentation, updates with confidence

Separating training and inference; closing the loop

- e.g., collecting telemetry to learn from user interactions

Learn, serve, and observe at scale or with resource limits

- e.g., cloud deployment, embedded devices

Scoping Relevant Qualities of ML Components

From System Quality Requirements to Component Quality Specifications

Design Decision: ML Model Selection

How do I decide which ML algorithm to use for my project?

Criteria: Quality Attributes & Constraints

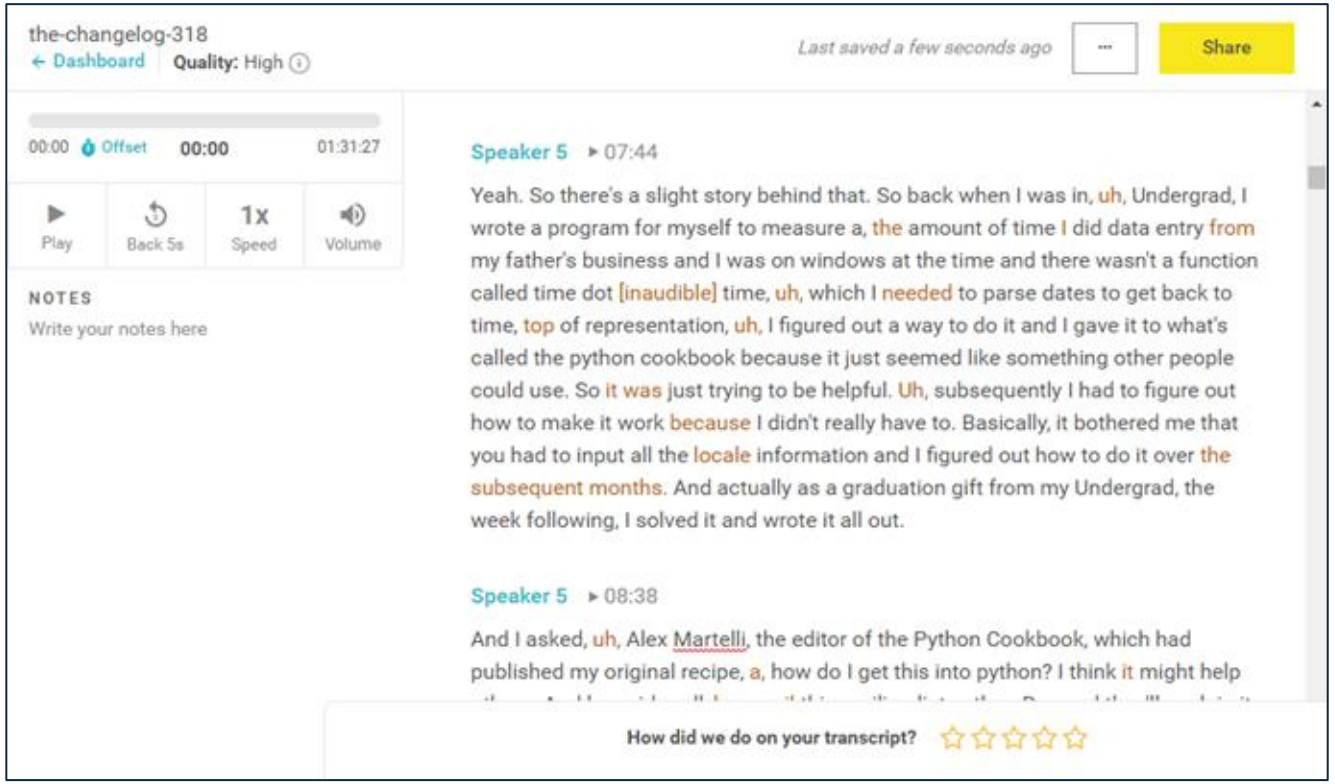
Accuracy is not Everything

Beyond prediction accuracy, what qualities may be relevant for an ML component?



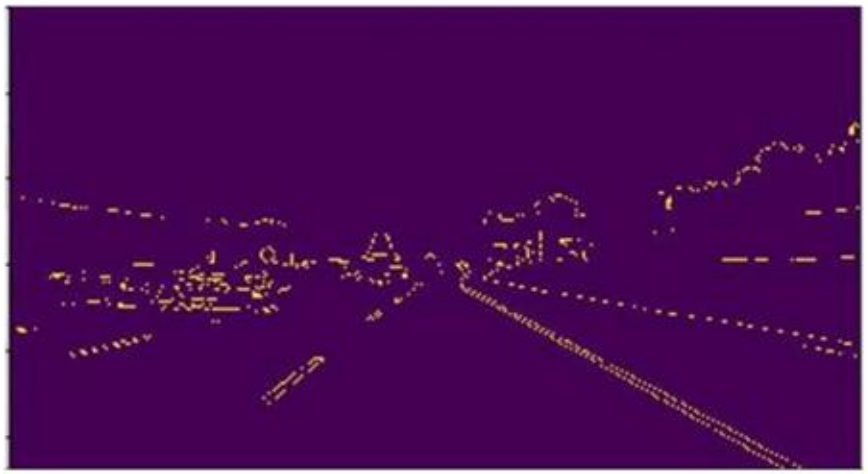
Qualities of Interest?

Scenario: ML component for transcribing audio files

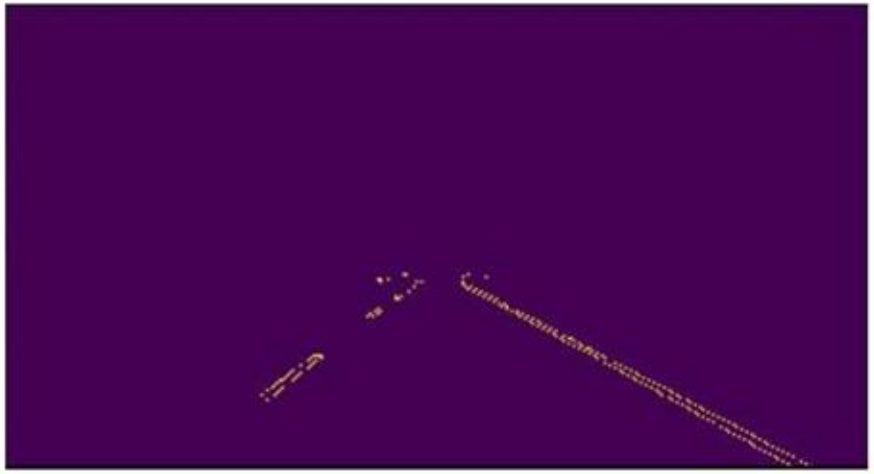


Qualities of Interest?

Scenario: Component for detecting lane markings in a vehicle



Canny edge detection output



Hough transform output

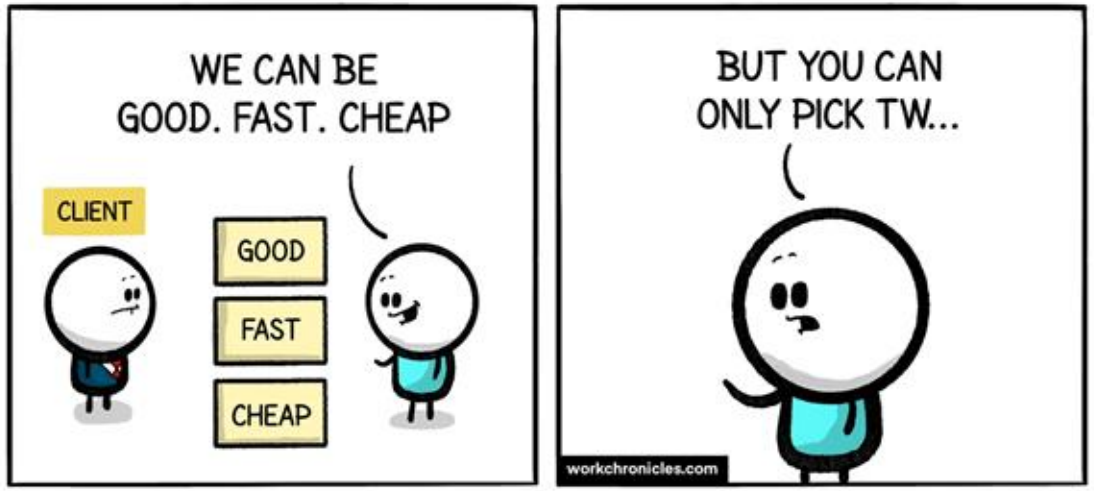
Qualities of Interest?

Scenario: Component for detecting credit card frauds, as a service for banks



Common ML Qualities to Consider

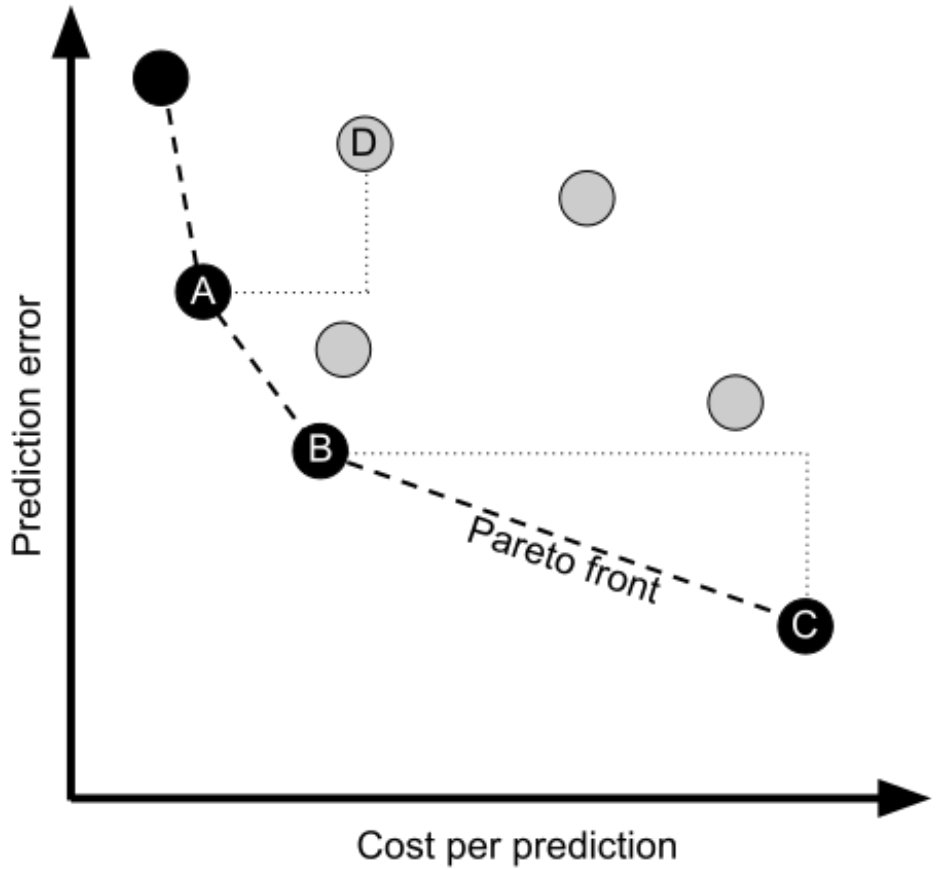
- Accuracy
- Correctness guarantees? Probabilistic guarantees (--> symbolic AI)
- How many features?
- How much data needed? Data quality important?
- Incremental training possible?
- Training time, memory need, model size -- depending on training data volume and feature size
- Inference time, energy efficiency, resources needed, scalability
- Interpretability, explainability
- Robustness, reproducibility, stability
- Security, privacy, fairness



Hello. I make comics about work.
Join r/workchronicles or follow on Instagram/Twitter/FB

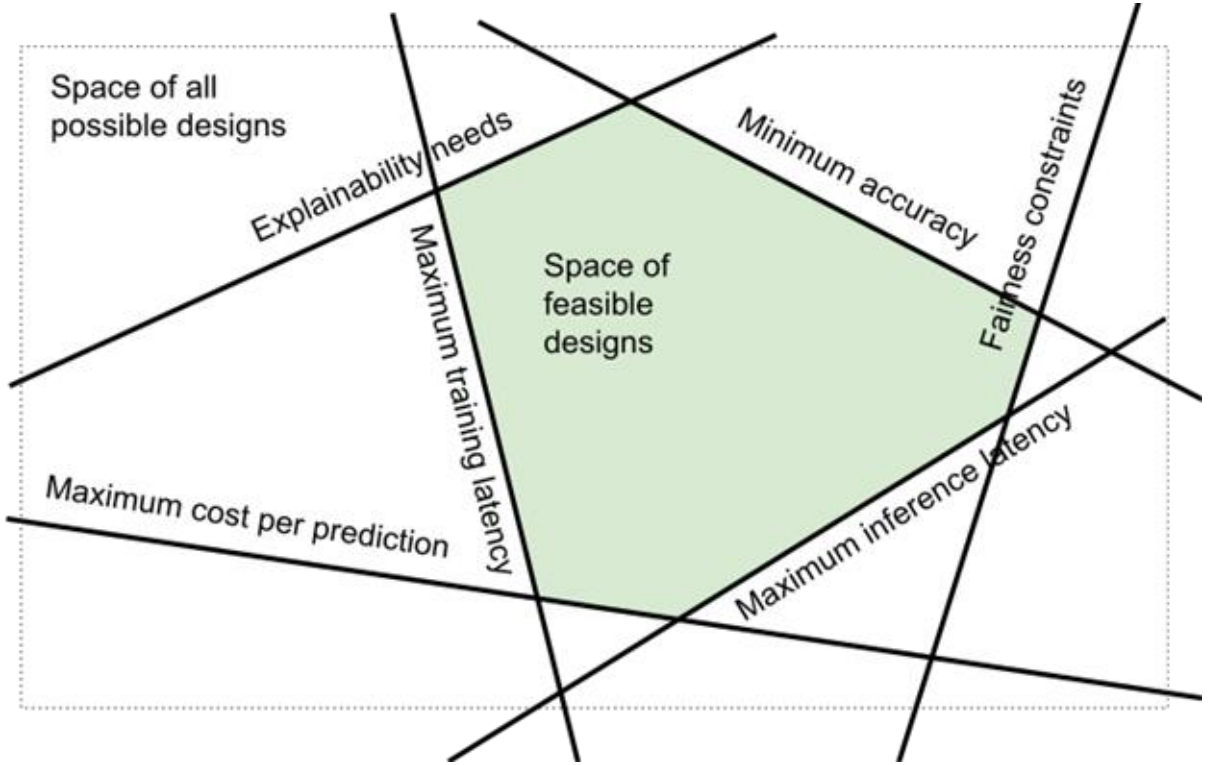
Work Chronicles
workchronicles.com

Constraints and Tradeoffs



Constraints

Constraints define the space of attributes for valid design solutions



Types of Constraints

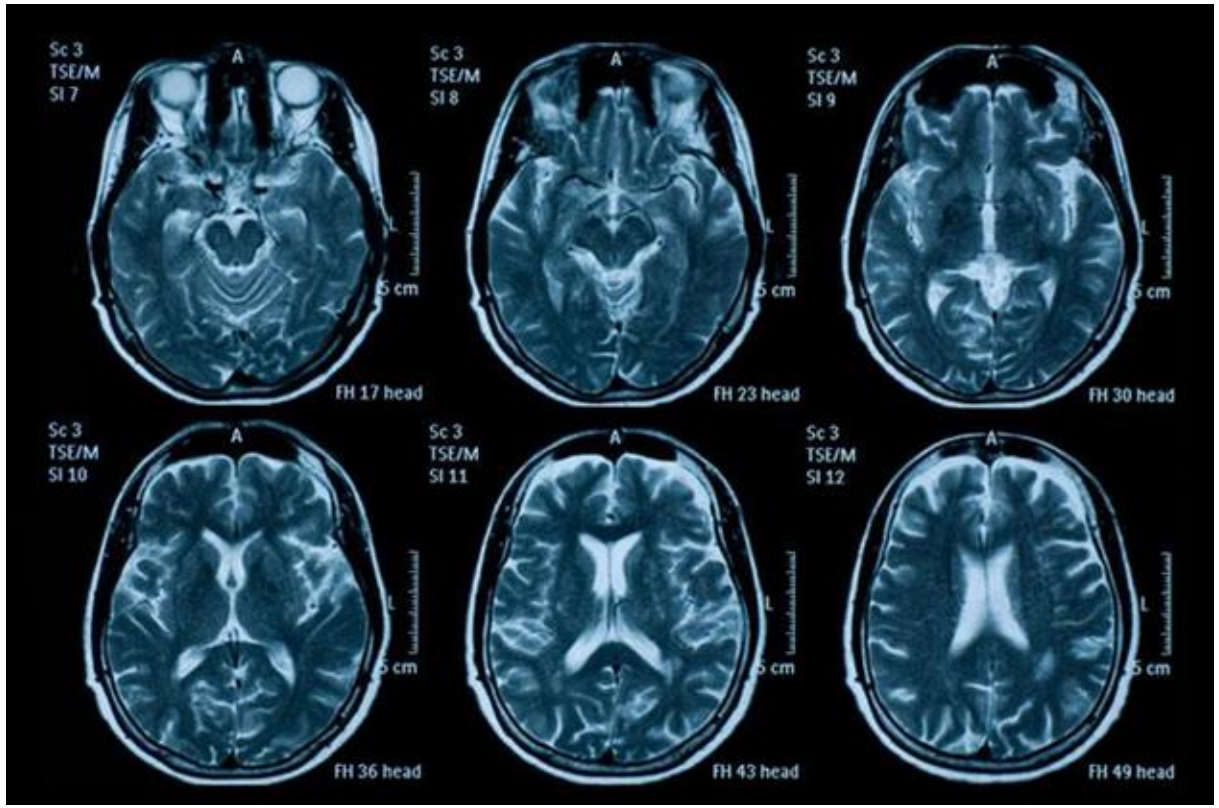
Problem constraints: Minimum required QAs for an acceptable product

Project constraints: Deadline, project budget, available personnel/skills

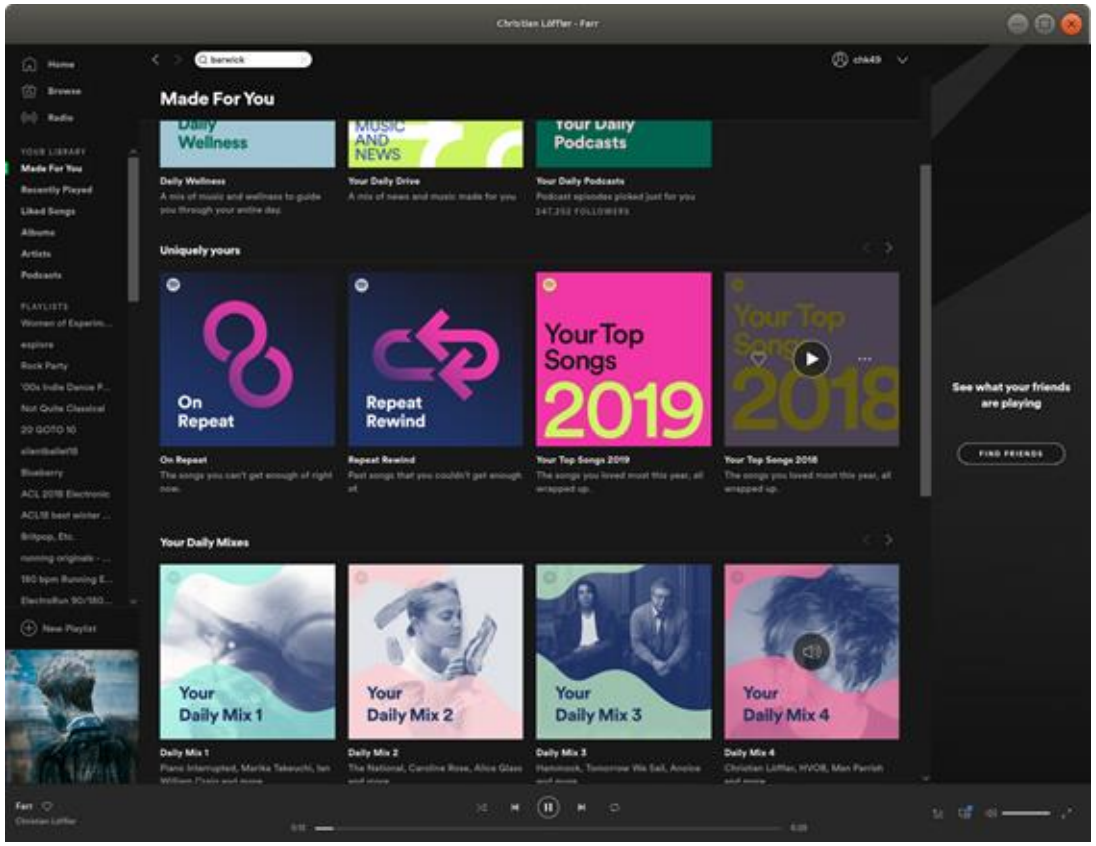
Design constraints

- Type of ML task required (regression/classification)
- Available data
- Limits on computing resources, max. inference cost/time

Constraints: Cancer Prognosis?



Constraints: Music Recommendations?



Trade-offs between ML algorithms

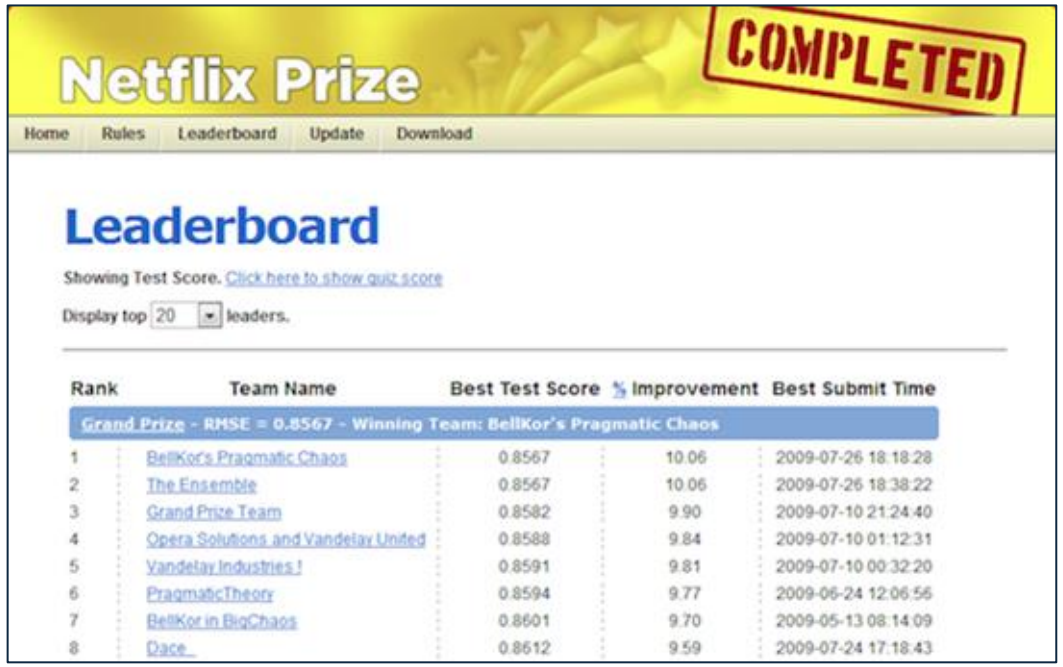
If there are multiple ML algorithms that satisfy the given constraints, which one do we select?

Different ML qualities may conflict with each other; this requires making a **trade-off** between these qualities

Among the qualities of interest, which one(s) do we care the most about?

- And which ML algorithm is most suitable for achieving those qualities?
- (Similar to requirements conflicts)

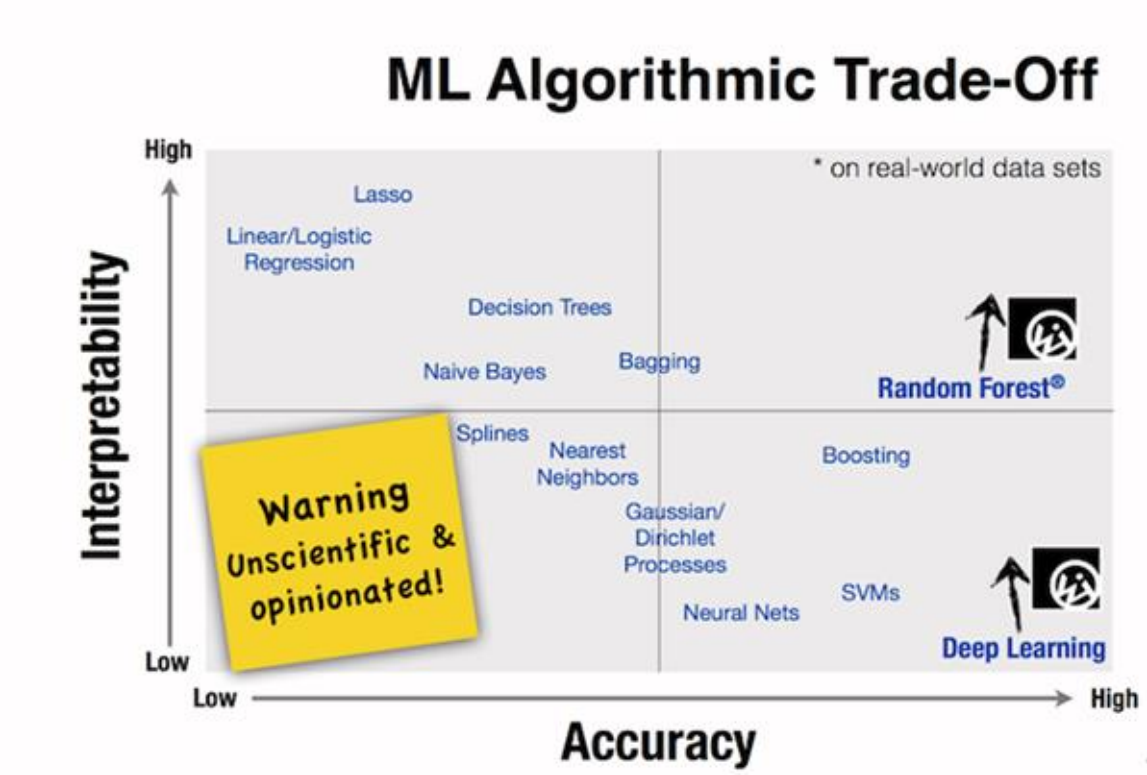
Trade-offs: Cost vs Accuracy



"We evaluated some of the new methods offline but the additional accuracy gains that we measured did not seem to justify the engineering effort needed to bring them into a production environment."

Amatriain & Basilico. Netflix Recommendations: Beyond the 5 stars, Netflix Technology Blog (2012)

Trade-offs: Accuracy vs Interpretability



Q. Examples where one is more important than the other?

Network Size

- 50 Layer ResNet network -- classifying 224x224 images into 1000 categories
  - 26 million weights, computes 16 million activations during inference, 168 MB to store weights as floats
- Google in 2012(!): 1TB-1PB of training data, 1 billion to 1 trillion parameters
- OpenAI's GPT-2 (2019) -- text generation
  - 48 layers, 1.5 billion weights (~12 GB to store weights)
  - released model reduced to 117 million weights
  - trained on 7-8 GPUs for 1 month with 40GB of internet text from 8 million web pages
- OpenAI's GPT-3 (2020): 96 layers, 175 billion weights, 700 GB in memory, $4.6M in approximate compute cost for training
- OpenAI's GPT-4 (2023) took 6 months and $100M
- Deepseek?

Cost & Energy Consumption

Consumption	CO2 (lbs)
Air travel, 1 passenger, NY↔SF	1984
Human life, avg, 1 year	11,023
American life, avg, 1 year	36,156
Car, avg incl. fuel, 1 lifetime	126,000

Training one model (GPU)	CO2 (lbs)
NLP pipeline (parsing, SRL)	39
w/ tuning & experimentation	78,468
Transformer (big)	192
w/ neural architecture search	626,155

Cost & Energy Consumption

Model	Hardware	Hours	CO2	Cloud cost in USD
Transformer	P100x8	84	192	289–981
ELMo	P100x3	336	262	433–1472
BERT	V100x64	79	1438	3751–13K
NAS	P100x8	274,120	626,155	943K–3.2M
GPT-2	TPUv3x32	168	—	13K–43K
GPT-3			—	4.6M